

Reto 02 — RA5

Pipeline Big Data para datos IoT con arquitectura medallón

Antonio Ferrer Martínez

Big Data Aplicado · 2.ª evaluación

CIFP Carlos III · Cartagena · curso 2025-2026

1. Descripción del caso de uso IoT elegido

El sensor que tomo es un dispositivo ambiental de aula. Le llamo sensor_aula_01. Lo imagino atornillado en la pared, leyendo cuatro magnitudes cada 30 segundos durante la jornada lectiva (de 9:00 a 17:00, sobre unas 1000 muestras al día): temperatura, humedad, CO₂ y nivel de carga del propio aparato.

No tengo el sensor real. El generador es un script en Python que produce las lecturas con números aleatorios pero usando distribuciones gaussianas centradas en valores típicos de aula (temperatura $\sim N(23, 2)$, humedad $\sim N(55, 8)$, CO₂ $\sim N(620, 80)$). La batería se descarga linealmente de 100 a 15 a lo largo del día, con un poco de ruido. Al ser yo quien decide qué inyectar como error, puedo asegurar que la fase de calidad tenga algo concreto que detectar.

Elegí ambiental antes que industrial o energético por dos razones que no son la principal y una que sí lo es. Las dos primeras: los campos típicos de un sensor de este tipo encajan con lo que necesito, y los rangos físicos son obvios para escribir reglas. La tercera, que era la importante para mí, es que un sensor de aula me dejaba pensar el dataset como si fuera real (cuándo entra gente a la sala, qué pasa con el CO₂ en la última hora cuando ya nadie ventila, cuándo deberían disparar alertas). Eso me ayudó a diseñar el dashboard del final con un enfoque útil en lugar de quedarse en un "aquí pinto temperatura por hora".

2. Esquema de los datos generados

Cada lectura es una línea JSON, formato JSONL (también llamado JSON Lines, especificado informalmente en jsonlines.org). Una línea, un registro independiente. Es lo más cómodo para escribir streaming a disco y para que Spark lea en paralelo después.

```
{
  "event_id":    "evt_000123_a4b9c2",
  "device_id":  "sensor_aula_01",
```

```
"timestamp": "2026-04-30T09:00:00",  
"temperature": 23.45,  
"humidity": 55.20,  
"co2": 620,  
"battery": 91,  
"status": "OK"  
}
```

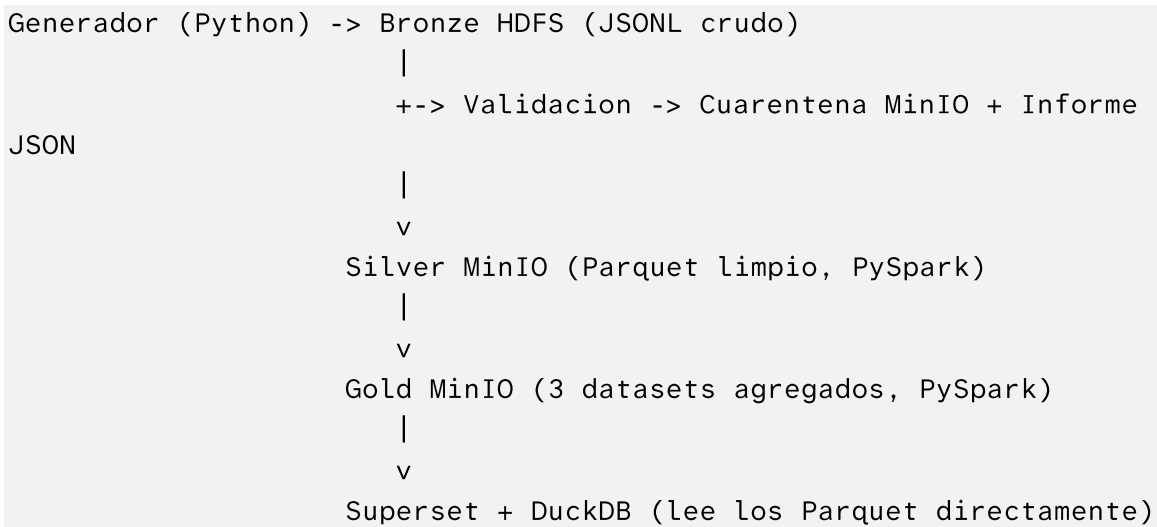
Ocho campos. Identificador del evento, identificador del dispositivo, marca temporal en formato ISO 8601 (sin sufijo Z), las cuatro métricas y el estado del aparato. El event_id no es sólo un contador: añadido un sufijo de seis hex sacados de un UUID4. Si por accidente ejecuto dos veces el generador con la misma semilla, los contadores chocan pero los hex no, así que sigue siendo único. El timestamp lo emito como string en lugar de tipo nativo porque la capa de calidad necesita verlo como llegó (cadena vacía vs null vs "no parseable"). El cast a TimestampType lo hace Spark mucho después.

Sobre el 7 % de los registros se inyectan con un fallo controlado, y son ocho tipos repartidos por las tres dimensiones de calidad. En precisión: temperature=150, humidity=-10, battery=130 y battery=-5. En completitud: timestamp="", timestamp con formato no ISO ("30/04/2026 10:15") y CO2 nulo. En consistencia: duplicados de event_id, clonando el registro entero como pasaría con un reintento de la cola IoT por timeout. Con esto cubro los tipos de error habituales que cabe esperar de un dispositivo IoT real: campos nulos, fechas mal formadas, valores fuera de rango físicamente plausible y duplicados por reintento. La proporción y la semilla son argumentos CLI (--records, --error-rate, --seed); si pongo --seed 42 los aleatorios son deterministas y el dataset es reproducible.

3. Diagrama o explicación de la arquitectura del pipeline

Tres capas (Bronze, Silver, Gold), patrón medallón, orquestadas por Airflow. Es una idea acuñada por Databricks (databricks.com/blog/what-is-medallion-architecture) que organiza el lago de datos en niveles de creciente calidad: Bronze guarda el dato crudo "in its original formats", Silver lo limpia y estandariza, Gold contiene

"business level aggregates and features ready for analytics". Yo lo implemento exactamente con esa lógica.



Bronze vive en HDFS porque el dato bruto es JSONL y replicarlo entre datanodes me sirve de seguro. Silver y Gold viven en MinIO en formato Parquet con compresión snappy. La transformación entre capas la hace Spark en modo local; a 1000 filas/día es ridículo montar un cluster, pero el código que escribo ya es el que tendría que ejecutarse en uno mañana, así que no pierdo nada.

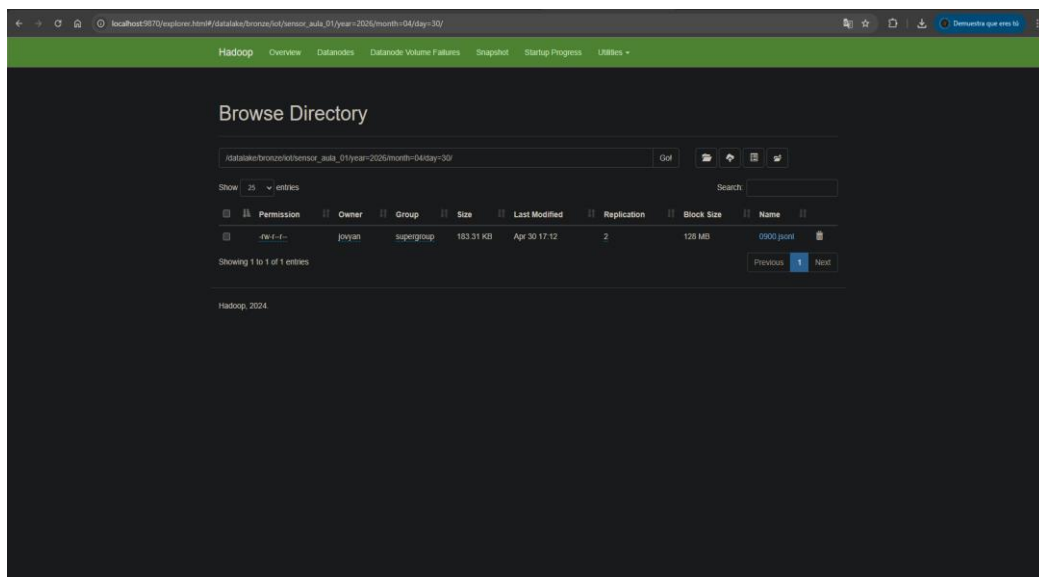
La pieza menos obvia del montaje fue cómo invocar Spark desde Airflow. La imagen oficial `apache/airflow:2.8.3` no trae Java ni PySpark, así que tenía dos opciones: añadirle Java + Spark a la imagen (la engorda 1,5 GB) o llamar al contenedor `jupyter-aula` con `docker exec` porque ese sí los tiene. Me decanté por la segunda. Sólo tuve que instalar el cliente Docker en la imagen extendida de Airflow y montar `/var/run/docker.sock`. Las dos tareas Spark del DAG son `BashOperators` con la forma `"docker exec jupyter-aula spark-submit ..."`. La primera vez que lo probé no funcionaba porque el contenedor `airflow-init` falla si lo arrancas como `root` (lo cuento en el apartado 10), y tardé un buen rato en ver que se podía dejar el `user`: `root` sólo en `webserver` y `scheduler`.

4. Captura de los datos Bronze en HDFS

Ruta con particionado Hive estándar:

```
/datalake/bronze/iot/sensor_aula_01/year=2026/month=04/day=30/0900.jsonl
```

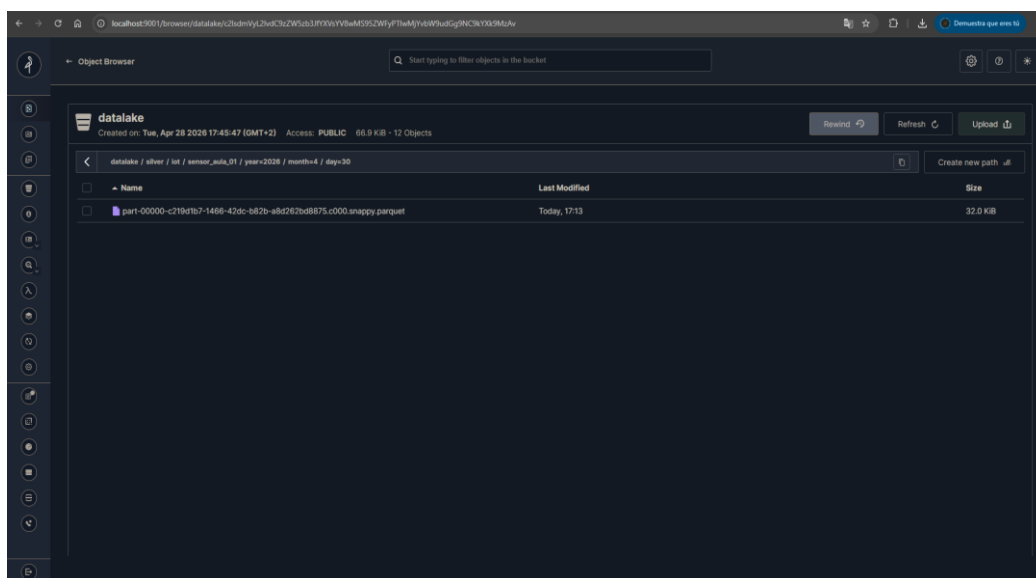
El script `load_bronze_hdfs.py` copia el JSONL local a HDFS sin abrirlo. Esa es la regla de Bronze: dato crudo, sin tocar. Si Silver o Gold se rompen aguas abajo, vuelvo aquí, leo y reproceso. El cliente que uso es el de la librería `hdfs` de Python (clase `InsecureClient`), que habla por WebHDFS al puerto 9870. Probé al principio con la CLI `hdfs dfs -put`, pero el contenedor de Airflow no trae los binarios de Hadoop, así que ese camino se descartó solo.



Bronze HDFS: 0900.jsonl (183 KB), replicación 2

5. Captura de los datos Plata y Oro en MinIO

Silver es Parquet snappy en `s3a://datalake/silver/iot/sensor_aula_01/`, particionado `year/month/day`. El script `bronze_to_silver.py` lee Bronze, descarta filas con timestamp inválido, filtra los rangos físicos, deduplica por `event_id` y añade columnas `year/month/day/hour`. Resultan 952 filas (de las 1000 válidas; los duplicados ya cayeron en `dropDuplicates`).



Silver: part-00000-...snappy.parquet (32 KiB)

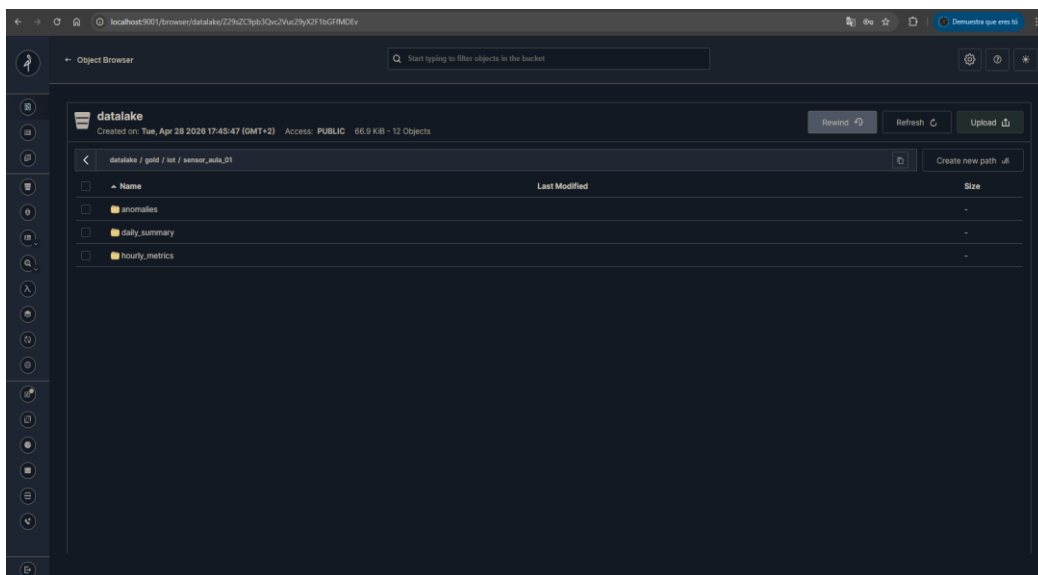
Parquet es el formato estándar para data lakes analíticos: columnar, comprime mejor que CSV y permite predicate pushdown. Sobre snappy frente a gzip o zstd, la documentación oficial de Apache Parquet recomienda snappy como balance velocidad/tamaño para cargas analíticas. Spark lee los Parquet con velocidad casi de RAM porque sólo deserializa las columnas que la query necesita.

Gold también es Parquet en MinIO, pero divididos en tres datasets. La razón de partirlos es que cada chart del dashboard pide datos en una granularidad distinta, y precomputar los agregados ahorra mucho trabajo a Superset:

- `hourly_metrics`: una fila por hora con avg/min/max de las cuatro métricas, conteo de eventos y métricas de calidad (`n_valid_events`, `n_invalid_events`, `pct_valid`, `n_alerts`). Los inválidos se cuentan leyendo la cuarentena, así el

dashboard puede mostrar "porcentaje de registros válidos por hora" sin un job extra.

- **daily_summary**: una fila por día con las medias, la batería mínima y los mismos contadores de calidad y anomalías que en **hourly_metrics**. La fecha del invalido la saco del directorio de cuarentena, no del timestamp interno, así no pierdo los registros que tenían precisamente el timestamp roto.
- **anomalies**: un registro por evento sospechoso (CO₂ alto, temperatura fuera de la zona de confort, batería por debajo del 20 %, etc.) etiquetado con su **anomaly_type**.



Gold: tres carpetas (anomalies, daily_summary, hourly_metrics)

6. Explicación de las reglas de calidad aplicadas

Las reglas viven en `src/quality/rules_iot.yaml`, separadas del código de validación.

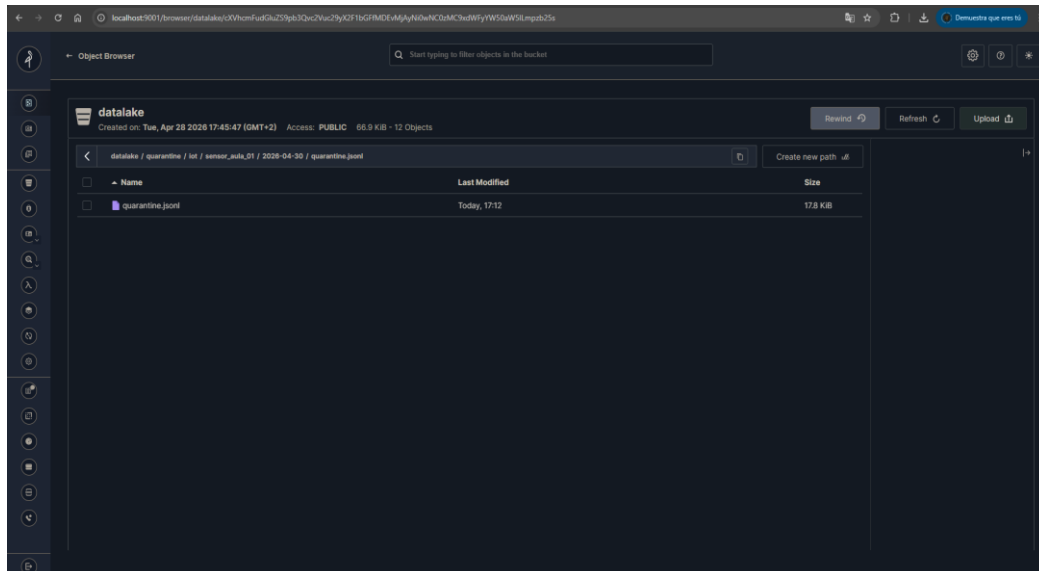
La idea, recogida también en Wang y Strong (1996), es que la calidad se descompone en dimensiones, y conviene tenerlas declaradas como dato (no como código) para poder ajustar umbrales sin tocar lógica. Mantener las reglas en un fichero YAML me permite, por ejemplo, mover el rango de temperatura sin desplegar de nuevo el contenedor. Bloque del fichero:

```
schema:
  required: [event_id, device_id, timestamp, temperature,
            humidity, co2, battery, status]
ranges:
  temperature: {min: -20, max: 80, dimension: precision}
  humidity:    {min: 0, max: 100, dimension: precision}
  co2:         {min: 0, max: 5000, dimension: precision}
  battery:     {min: 0, max: 100, dimension: precision}
timestamp:
  field: timestamp
  format: "%Y-%m-%dT%H:%M:%S"
  dimension: completitud
unique:
  - field: event_id
    dimension: consistencia
```

Cada regla está etiquetada con la dimensión que cubre. La división clásica de tres dimensiones es: precisión, completitud y consistencia. Wang y Strong publican en 1996 ("Beyond Accuracy", Journal of Management Information Systems 12(4)) una taxonomía más amplia con cuatro categorías (intrínseca, contextual, representacional, accesibilidad), pero para este proyecto las tres clásicas cubren los casos que interesan. Si una temperatura llega a 150, falla en precisión. Si el timestamp es vacío, falla en completitud. Si su event_id está duplicado, falla en consistencia.

Los registros que no pasan no se borran. Se aíslan en una zona aparte (cuarentena) en `s3://datalake/quarantine/iot/sensor_aula_01/<fecha>/quarantine.jsonl`. Cada registro inválido conserva sus campos originales y se le añade uno extra,

_quality_errors, con la lista de motivos. Esto es el patrón Dead Letter Queue clásico de mensajería empresarial llevado a un pipeline de datos. Sirve para auditoría: puedo ir más adelante y ver exactamente qué tipo de fallos son los más frecuentes.



quarantine.jsonl en MinIO con los registros descartados

7. Informe de calidad generado por el pipeline

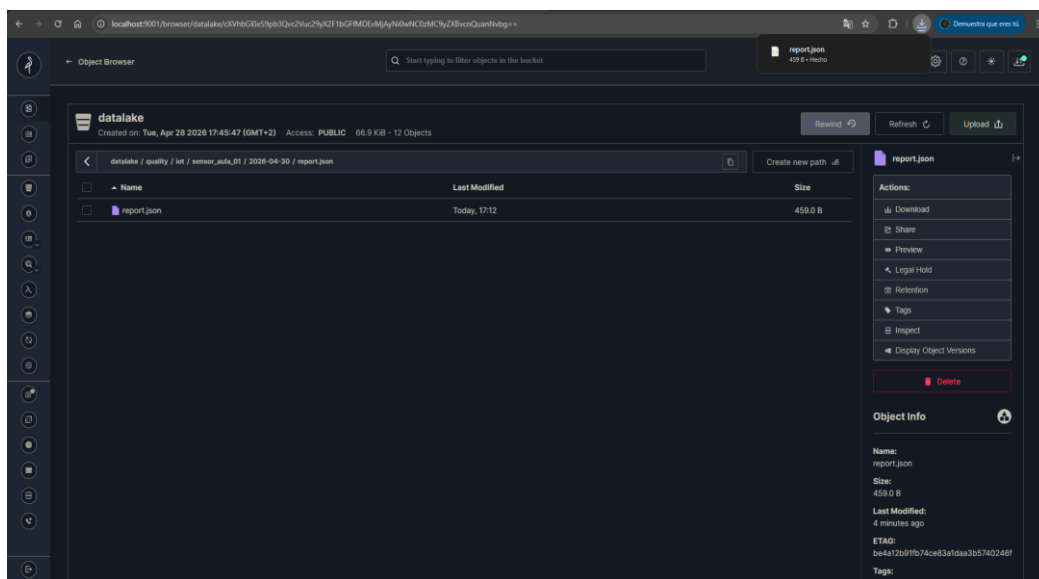
Cada ejecución produce un report.json en

s3://datalake/quality/iot/sensor_aula_01/<fecha>/. Lo escribe la propia tarea de validación al terminar, y queda accesible desde la consola web de MinIO (puerto 9001) sin necesidad de descargar nada. Este es el informe del run del 30 de abril:

```
{
  "device_id": "sensor_aula_01",
  "run_date": "2026-04-30",
  "generated_at_utc": "2026-04-30T15:12:36+00:00",
  "total_records": 1016,
  "valid_records": 936,
  "invalid_records": 80,
  "by_rule": {
    "humidity_out_of_range": 13,
    "battery_out_of_range": 14,
    "temperature_out_of_range": 10,
    "duplicate_event_id": 32,
    "missing_timestamp": 11
  },
  "by_dimension": {
    "precision": 37,
    "consistencia": 32,
    "completitud": 11
  }
}
```

Lectura: 936 de 1016 registros (92,1 %) pasaron todas las reglas. Los 80 inválidos se reparten entre las tres dimensiones con números coherentes con la inyección que hace el generador. El bloque by_rule rompe el dato a nivel de regla concreta, y by_dimension lo agrega a alto nivel; me parece que cualquiera de las dos vistas es útil dependiendo de quién mire el informe.

Si el porcentaje de inválidos pasara del 50 %, el script aborta con RuntimeError y la tarea Airflow marca el run como failed. Lo añadí después de pensar en qué pasaría si una mañana el sensor empieza a mandar basura sistemática: no quiero que el pipeline siga procesando y meta los datos rotos en Silver y Gold tan tranquilo.



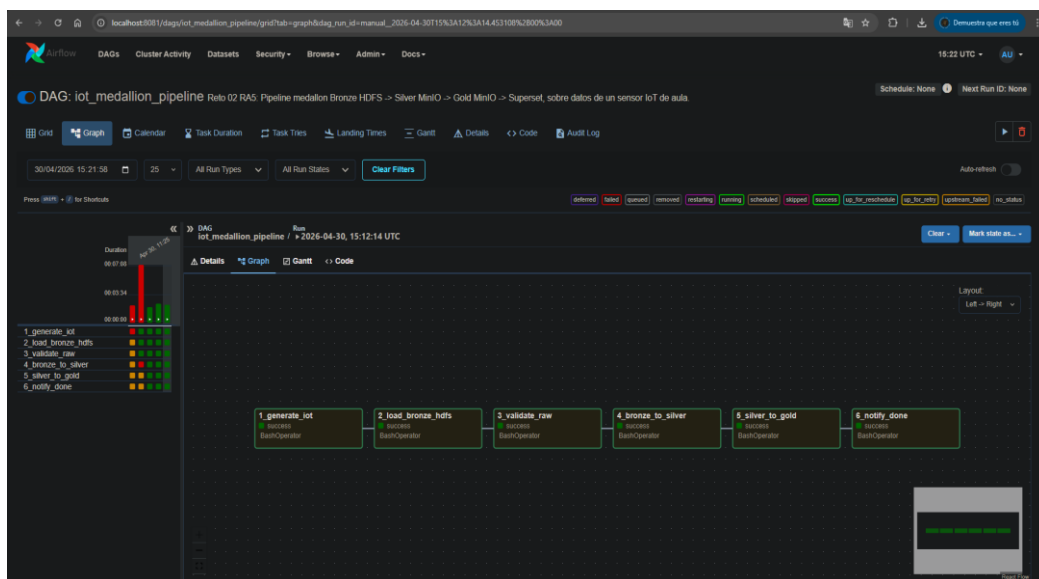
Carpeta quality/iot/sensor_aula_01/<fecha>/ con report.json

8. Captura del DAG de Airflow ejecutado correctamente

El DAG `iot_medallion_pipeline` encadena las ocho fases en serie. La cadena es lineal y refleja el flujo natural de un pipeline medallón:

```
generate_iot
-> validate_raw_format      (formato JSON antes de tocar HDFS)
   -> load_bronze_hdfs
       -> validate_quality  (reglas, cuarentena, informe)
           -> bronze_to_silver
               -> validate_silver
                   -> silver_to_gold
                       -> publish_gold_superset
```

Las ocho tareas son `BashOperator` que invocan scripts Python reales. Las dos tareas Spark se delegan al contenedor `jupyter-aula` con `docker exec spark-submit`, como conté en el apartado 3. El resto corre directamente en el contenedor de Airflow.



Las 8 tareas del DAG en estado success

El log de la tarea `bronze_to_silver` demuestra que no es un `BashOperator` de adorno: se ve la cabecera "Spark 3.5.0", los mensajes INFO de `SparkContext` y `MapOutputTrackerMasterEndpoint`, una tabla con datos reales del sensor y el cierre limpio "OK bronze_to_silver finalizado". Confirma que Spark arranca, descarga jars de Maven la primera vez, lee de HDFS y escribe a MinIO.



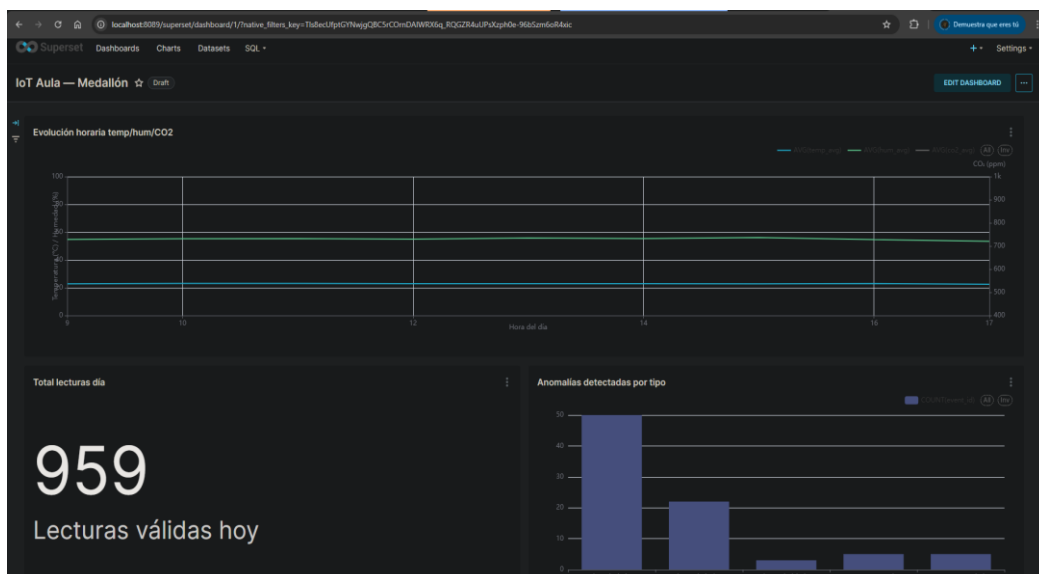
Log de bronze_to_silver con la salida real de Spark

9. Captura del dashboard creado en Superset

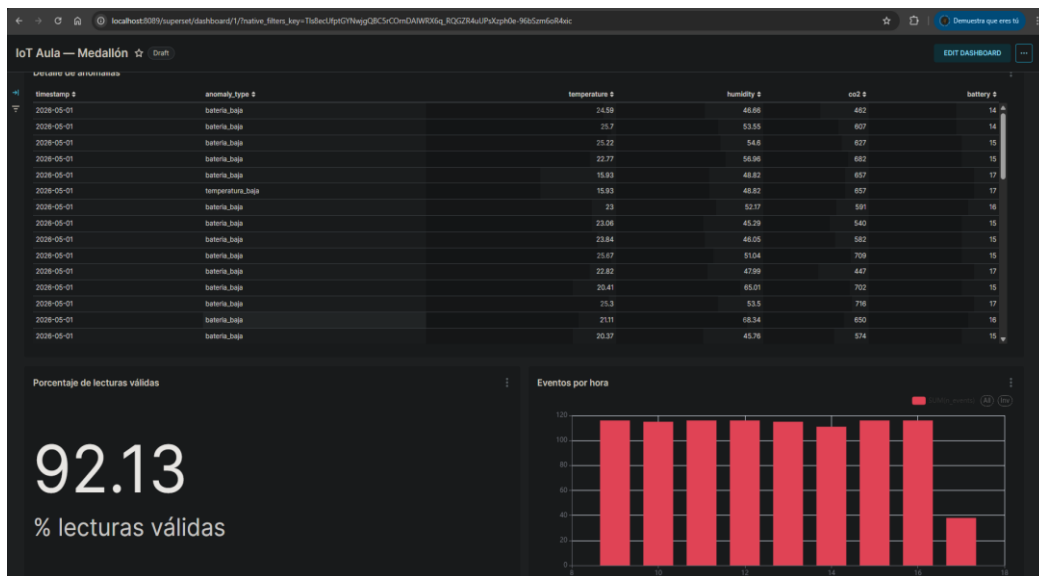
Superset se conecta a MinIO con DuckDB y la extensión httpfs. DuckDB es una base analítica embebida (Raasveldt y Mühleisen, "DuckDB: an Embeddable Analytical Database", SIGMOD 2019) que puede leer Parquet directamente desde S3 con predicate pushdown nativo. La gracia es que no necesito un ETL intermedio que copie los Parquet a una base relacional como PostgreSQL; Superset consulta el dato fresco directamente. La conexión se configura con este JSON de engine_params:

```
{
  "connect_args": {
    "preload_extensions": ["httpfs"],
    "config": {
      "s3_endpoint": "minio:9000",
      "s3_access_key_id": "admin",
      "s3_secret_access_key": "adminadmin",
      "s3_url_style": "path",
      "s3_use_ssl": false
    }
  }
}
```

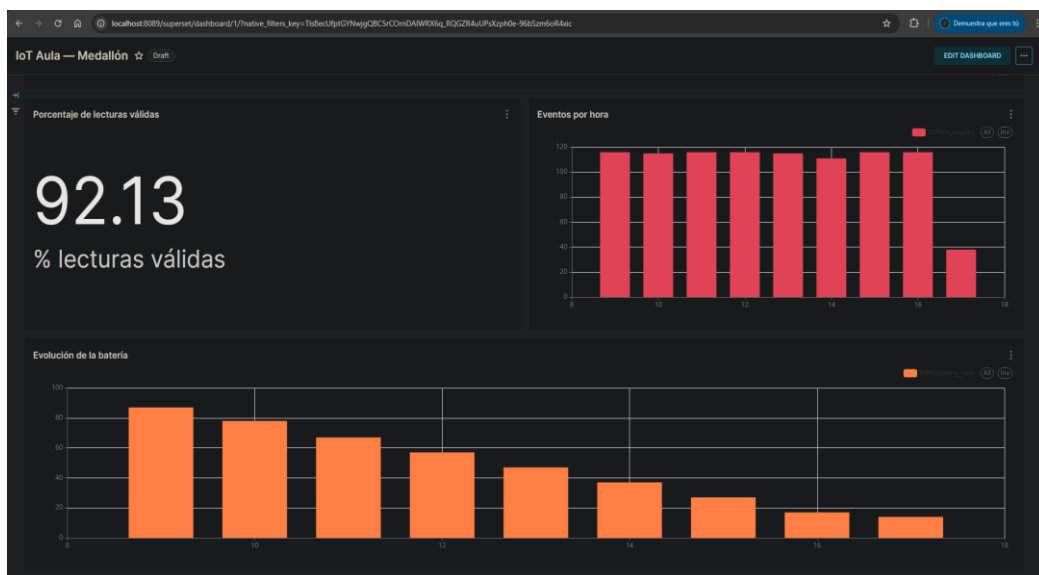
El dashboard se llama "IoT Aula — Medallón" y reúne siete charts. Lo enseño en tres bloques (parte de arriba, parte central y parte de abajo) porque entero no entra en una pantalla normal:



Dashboard: cabecera con la evolución horaria de temp/humedad/CO₂

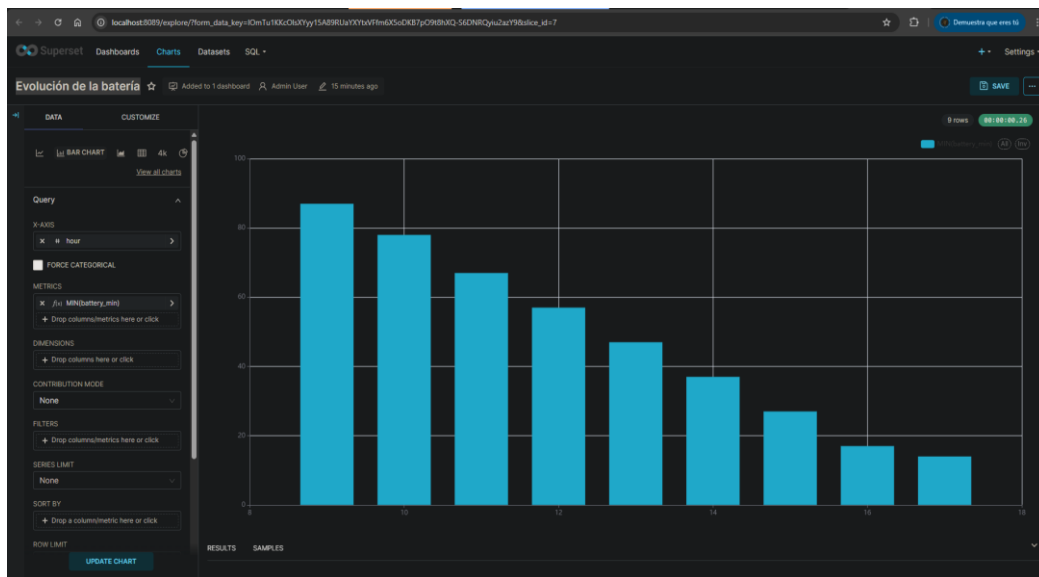


Dashboard: tabla de detalle de anomalías

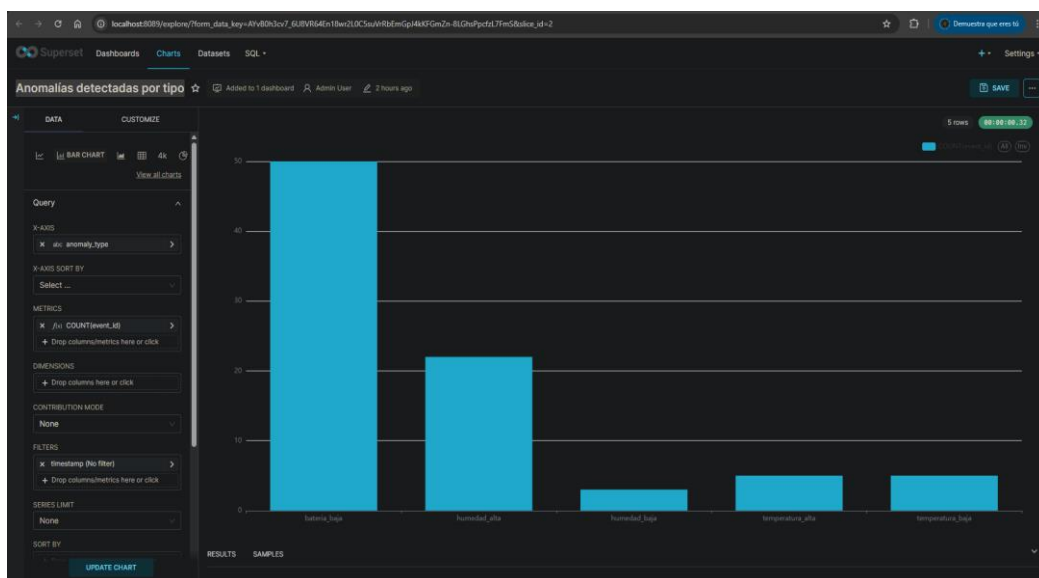


Dashboard: porcentaje de válidos, eventos por hora y evolución de la batería

Hay dos charts que destaco aparte por separado porque son los que cambiaron entre la primera versión y la final. La curva de batería empezó siendo plana (en el primer generador la batería era `random.randint(70, 100)` y se quedaba toda igual) y la cambié a descarga lineal después de ver que el chart no contaba nada. Ahora baja claramente de 88 a 14 a lo largo del día y cruza el umbral del 20 % en la última franja, lo que dispara las anomalías de tipo `bateria_baja`:



Evolución de la batería: curva descendente real



Anomalías por tipo: batería_baja domina al final del día

Lectura del dashboard: la temperatura del aula se mantiene estable alrededor de 23 °C durante todo el día, dentro de la zona de confort. La humedad oscila entre 50 y 60 %. El CO₂ ronda los 615 ppm, claramente bajo. Sobre este último valor descubrí algo investigando para esta memoria: la cifra de 1000 ppm de CO₂ que se cita habitualmente como umbral de "ventilación insuficiente" se asocia mucho a ASHRAE 62.1, pero ASHRAE no incluye ese límite en su estándar desde hace casi 30 años, y de hecho la propia ASHRAE publica un position document de 2022 llamado "Indoor Carbon Dioxide" (consultable en [ashrae.org](https://www.ashrae.org)) para aclarar el malentendido. El umbral que sigo usando (1000 ppm) procede de la práctica

común de la industria y de varias guías de calidad de aire interior, pero formalmente no es un requisito ASHRAE. Lo dejo claro aquí porque me parece más honesto que repetirlo como si fuera norma.

La batería cae sostenidamente hasta cruzar el 20 % en la última hora, lo que dispara la mayoría de las anomalías. La calidad del lote es alta: 92,1 % de lecturas válidas. Los rechazos se reparten entre las tres dimensiones sin que ninguna domine de forma anómala, lo cual es señal de que el inyector y el validador están trabajando como se espera.

10. Problemas encontrados y soluciones aplicadas

Recojo los siete fallos técnicos más relevantes que tuve durante el desarrollo, con la causa que diagnosticué y cómo los resolví. La mayoría me costaron más de un intento.

1) Spark fallaba con `java.lang.ClassNotFoundException`:

`org.apache.hadoop.fs.s3a.S3AFileSystem` al intentar escribir a MinIO. Lo gracioso es que la JVM levantaba bien y el `SparkContext` se creaba sin problema; el error sólo aparecía en el primer write a `s3a://`. Diagnóstico: la propiedad `spark.jars.packages` dentro de `SparkSession.builder` no se aplica antes de inicializar la JVM; cuando Python crea la sesión la JVM ya está arrancada y los paquetes llegan tarde. La documentación oficial del módulo `hadoop-aws` (hadoop.apache.org, sección "Troubleshooting S3A") advierte además que las versiones de `hadoop-common` y `hadoop-aws` deben ser exactamente iguales y que cambiar el AWS SDK por encima de lo que Hadoop fue compilado sólo cambia el stack trace, no arregla nada. La combinación que me funciona con Spark 3.5 / Hadoop 3.4 es `hadoop-aws:3.3.4 + aws-java-sdk-bundle:1.12.262`, pasada como `--packages` directamente al `spark-submit`.

2) El contenedor `airflow-init` no arrancaba: `ModuleNotFoundError: No module named airflow`. Estaba ejecutándose como `root` porque mi primera versión del `docker-compose.override.yml` ponía `user: root` en TODOS los servicios de Airflow para que pudieran usar el socket Docker. Pero el paquete `airflow` se instala en `/home/airflow/.local/`, accesible sólo para el usuario `airflow`. Lo arreglé dejando `user: root` únicamente en `webserver` y `scheduler` (que sí necesitan llamar a `docker exec`) y devolviendo `airflow-init` a su usuario por defecto.

3) Al construir la imagen extendida de Airflow, el `RUN pip install --target /opt/extra-libs` petó con "Can not combine `--user` and `--target`". Tardé un rato en pillar que la imagen oficial define la variable `PIP_USER=true` por defecto, lo que mete `--user`

implícitamente en cada pip install. Lo arreglé prefijando el comando con `PIP_USER=no` para anular el default sólo durante esa instalación.

4) Los volúmenes nuevos del `docker-compose.override.yml` no aparecían tras un `--force-recreate`. Compose no reaplica los cambios del override sobre contenedores existentes; hay que hacer `down --remove-orphans` completo y un `up` limpio. Lo descubrí mirando `docker inspect` del scheduler y viendo que sólo tenía los volúmenes del compose original, no los míos.

5) `"import pyspark"` no funciona dentro de jupyter-aula aunque Spark esté instalado en `/usr/local/spark`. La imagen `jupyter/pyspark-notebook` trae Spark como aplicación (con `spark-submit` en el PATH) pero no como paquete pip, así que el módulo `pyspark` vive en `/usr/local/spark/python/`, fuera del `PYTHONPATH` por defecto. Llamando con `spark-submit` en lugar de `python`, el binario configura `PYTHONPATH` y el classpath de Java automáticamente y todo va.

6) En el primer chart de evolución horaria, las líneas de temperatura y humedad se aplastaban contra el eje X. El problema era de escalas: las tres métricas comparten un eje Y, pero CO_2 está alrededor de 600 mientras temperatura está en 23 y humedad en 55. La línea de CO_2 ocupa toda la altura visible y deja a las otras dos pegadas al suelo. Lo arreglé cambiando el chart a Mixed Chart y separando los ejes: temperatura y humedad en el eje izquierdo (escala 0-100), CO_2 en el derecho (escala 400-1000). Es el tipo de cosa que se aprende viendo el chart pintado: en abstracto no se nota, en pantalla sí.

7) Con 8 GB de RAM no puedo levantar todos los servicios a la vez. El entorno define varios profiles (`core`, `batch`, `orchestration`, `bi`, `monitoring`) precisamente para alternarlos. Para el pipeline uso `core+batch+orchestration` y para la visualización uso `core+bi`. Entre cambios un `docker compose down --remove-orphans` cierra el perfil anterior. MinIO conserva sus volúmenes persistentes entre cambios, así que Superset siempre encuentra el último Gold escrito.